

GSOC 2025 Proposal

CEBRA-Based Data Processing Pipeline for Mapping Time-Locked EEG Paired Sets in Interacting Participants

ML4SCI Organization

Personal Information

Name: Harshit Singh
Email:  bciandcoding@gmail.com
Colab:  [Evaluation Task](#)
GitHub:  github.com/BCImonk
LinkedIn:  [harshit-singh-bciandcoding](#)

Project Summary

This project proposes developing a computational pipeline leveraging CEBRA (Contrastive Embedding for Behavioral and Neural Analysis) to analyze time-locked EEG data from interacting participants. The social neuroscience community has long sought objective neural markers that can characterize differences in social communication, particularly in conditions like Autism Spectrum Disorder (ASD). As highlighted by Hasson et al. (2022), current measurement approaches often fail to capture the complex, non-linear dynamics of neural synchrony during social interactions.

The proposed pipeline addresses this gap by applying advanced computational methods to extract meaningful patterns from dyadic EEG recordings. By building on my background in Brain-Computer Interfaces and machine learning, this project aims to develop tools that could lead to better understanding of neurological conditions through objective neural markers, potentially transforming how we diagnose and develop interventions for social communication disorders.

Background and Motivation

The Scientific and Societal Need

Social communication deficits represent a central challenge across multiple clinical conditions, with profound impacts on quality of life, educational outcomes, and employment opportunities. Currently, diagnosis of these conditions relies heavily on behavioral observations and subjective assessments, creating challenges for early intervention. The groundbreaking work by Dikker et al. (2021) demonstrated that neural synchrony during social interactions can be measured with EEG and serves as a potential biomarker for social engagement.

However, accurately measuring the complex, non-linear neural dynamics during social interactions remains challenging. As Hasson and Frith (2016) emphasized in their influential paper on neural synchronization, current methods primarily use correlation-based approaches that fail to capture the rich temporal dynamics of neural coupling. This limitation hampers our ability to develop precision diagnostics and targeted interventions.

Why CEBRA?

Schneider et al.'s (2023) introduction of CEBRA offers a promising approach for analyzing neural data in a novel way. Their work demonstrated how contrastive learning can extract meaningful latent representations from complex neural recordings, establishing relationships between neural activity and behavior. Building on this foundation, we can adapt CEBRA to analyze paired neural recordings—a novel application with significant potential.

Recent work by Sareen et al. (2023) on time-resolved modulation of directed coupling between brains highlights the need for sophisticated computational methods to understand these complex interactions. CEBRA provides unique advantages for this application:

1. Ability to extract *meaningful, low-dimensional representations* from complex EEG data
2. Superior tools for visualizing *hidden patterns in neural communication*
3. Potential to reveal *subtle differences between neurotypical and clinical populations*

As emphasized by Papadopoulou et al. (2019), estimating directed connectivity from cortical recordings requires methods that can handle non-linear relationships and temporal dynamics—requirements that CEBRA is uniquely positioned to meet.

Technical Approach

Data Preprocessing

Building on established methods in the field and previous experience with data preprocessing:

1. EEG Signal Preprocessing:

- Apply ICA methods to remove eyeblinks and other artifacts
- Implement bandpass filtering to isolate relevant frequency bands
- Epoch data around speaking/listening events
- Develop time-locking procedures between participant pairs

```
1 # ICA-based artifact removal
2 import mne
3 from mne.preprocessing import ICA
4
5 def preprocess_eeg(raw_eeg):
6     # Apply bandpass filter (1-40 Hz)
7     filtered_eeg = raw_eeg.filter(l_freq=1, h_freq=40, method='fir', fir_design='firwin')
8
9     # Apply ICA for artifact removal
10    ica = ICA(n_components=20, random_state=42)
11    ica.fit(filtered_eeg)
12
13    # Auto-detect and remove eye blink artifacts
14    eog_indices, eog_scores = ica.find_bads_eog(filtered_eeg)
15    ica.exclude = eog_indices
16
17    # Apply ICA correction
18    cleaned_eeg = filtered_eeg.copy()
19    ica.apply(cleaned_eeg)
20
21    return cleaned_eeg
22
23 # Time-locking between participant pairs
24 def time_lock_dyads(speaker_eeg, listener_eeg, event_markers):
25     # Synchronize recordings based on event markers
26     paired_epochs = []
27     for event in event_markers:
28         speaker_epoch = mne.Epochs(speaker_eeg, events=event, tmin=-0.5, tmax=2.0)
29         listener_epoch = mne.Epochs(listener_eeg, events=event, tmin=-0.5, tmax=2.0)
30         paired_epochs.append((speaker_epoch, listener_epoch))
31
32     return paired_epochs
```

Listing 1: Preliminary code for EEG preprocessing using MNE-Python

2. Data Formatting:

- Transform preprocessed EEG data into CEBRA-compatible formats
- Create paired datasets for speaker-listener dyads
- Ensure proper alignment across participants

CEBRA Implementation

Following the theoretical framework established by Schneider et al. (2023) and integrating elements from Hölle et al.’s (2021) work on mobile EEG applications:

1. Model Architecture Adaptation:

- Modify CEBRA’s contrastive learning framework for paired EEG data
- Implement appropriate time-based conditioning variables
- Design loss functions that capture inter-brain relationships

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class DyadicCEBRA(nn.Module):
6     def __init__(self, input_dim=64, embedding_dim=32, conditioning_dim=10):
7         super(DyadicCEBRA, self).__init__()
8         # Encoders for speaker and listener
9         self.speaker_encoder = nn.Sequential(
10             nn.Linear(input_dim, 128), nn.ReLU(),
11             nn.Linear(128, 64), nn.ReLU(),
12             nn.Linear(64, embedding_dim)
13         )
14
15         self.listener_encoder = nn.Sequential(
16             nn.Linear(input_dim, 128), nn.ReLU(),
17             nn.Linear(128, 64), nn.ReLU(),
18             nn.Linear(64, embedding_dim)
19         )
20
21         # Conditioning network
22         self.conditioning = nn.Sequential(
23             nn.Linear(conditioning_dim, 32), nn.ReLU(),
24             nn.Linear(32, embedding_dim)
25         )
26
27     def forward(self, speaker_data, listener_data, condition):
28         speaker_embedding = self.speaker_encoder(speaker_data)
29         listener_embedding = self.listener_encoder(listener_data)
30         condition_embedding = self.conditioning(condition)
31
32         return speaker_embedding, listener_embedding, condition_embedding
33
34 # Custom loss function for neural synchrony
35 def neural_synchrony_loss(speaker_emb, listener_emb, condition_emb, temp=0.5):
36     # InfoNCE loss for contrastive learning
37     # Positive pairs: time-matched speaker-listener dyads
38     # Negative pairs: non-matched speaker-listener dyads
39     pos_similarity = F.cosine_similarity(speaker_emb, listener_emb, dim=1) / temp
40
41     # Batch-wise negative samples
42     speaker_neg = speaker_emb.repeat(speaker_emb.size(0), 1, 1)
43     listener_neg = listener_emb.unsqueeze(1).repeat(1, listener_emb.size(0), 1)
44
45     # Compute similarities for all negative pairs
46     neg_similarity = F.cosine_similarity(speaker_neg.view(-1, speaker_emb.size(-1)),
47                                         listener_neg.view(-1, listener_emb.size(-1)),
48                                         dim=1).view(speaker_emb.size(0), -1)
49
50     # Mask out positive pairs
51     mask = torch.eye(speaker_emb.size(0), device=speaker_emb.device)
52     neg_similarity = neg_similarity * (1 - mask) - mask * 1e9
53
54     # InfoNCE loss
55     logits = torch.cat([pos_similarity.unsqueeze(1), neg_similarity], dim=1)
56     labels = torch.zeros(speaker_emb.size(0), device=speaker_emb.device, dtype=torch.long)
57
58     return F.cross_entropy(logits, labels)
```

Listing 2: Proposed CEBRA architecture adaptation for dyadic EEG data

2. Training Pipeline:

- Develop within-dyad and across-dyad training procedures
- Implement cross-validation strategies for paired data
- Create visualization tools to interpret neural synchrony patterns

Analysis Framework

Building on the methodological approaches outlined by Papadopoulou et al. (2019) and Sareen et al. (2023):

1. Latent Space Analysis:

- Measure topological properties of learned manifolds
- Develop metrics to quantify neural synchrony in latent space
- Track temporal dynamics of embeddings during communication events

```
1 import numpy as np
2 from sklearn.manifold import TSNE
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy.stats import pearsonr
6
7 def analyze_latent_space(speaker_embeddings, listener_embeddings, labels):
8     # Dimensionality reduction for visualization
9     combined_embeddings = np.vstack([speaker_embeddings, listener_embeddings])
10    tsne = TSNE(n_components=2, random_state=42)
11    reduced_embeddings = tsne.fit_transform(combined_embeddings)
12
13    # Split back to speaker and listener
14    speaker_reduced = reduced_embeddings[:len(speaker_embeddings)]
15    listener_reduced = reduced_embeddings[len(speaker_embeddings):]
16
17    # Measure neural synchrony using correlation
18    correlations = []
19    for i in range(speaker_embeddings.shape[1]):
20        r, p = pearsonr(speaker_embeddings[:, i], listener_embeddings[:, i])
21        correlations.append((r, p))
22
23    # Calculate euclidean distances between paired embeddings
24    distances = np.sqrt(np.sum((speaker_embeddings - listener_embeddings)**2, axis=1))
25
26    # Visualize the latent space
27    plt.figure(figsize=(12, 10))
28
29    # Speaker embeddings
30    plt.scatter(speaker_reduced[:, 0], speaker_reduced[:, 1],
31               c=labels, cmap='viridis', marker='o', alpha=0.7, label='Speaker')
32
33    # Listener embeddings
34    plt.scatter(listener_reduced[:, 0], listener_reduced[:, 1],
35               c=labels, cmap='viridis', marker='x', alpha=0.7, label='Listener')
36
37    # Connect paired samples with lines
38    for i in range(len(speaker_reduced)):
39        plt.plot([speaker_reduced[i, 0], listener_reduced[i, 0]],
40                 [speaker_reduced[i, 1], listener_reduced[i, 1]],
41                 'k-', alpha=0.2)
42
43    plt.colorbar(label='Condition')
44    plt.legend()
45    plt.title('Latent Space Visualization of Speaker-Listener Dyads')
46
47    return correlations, distances
```

Listing 3: Example code for latent space analysis

2. Clinical vs. Neurotypical Comparison:

- Identify discriminative features in latent space
 - Test feature generalizability across different dyads
 - Connect findings to theoretical frameworks of social communication in ASD
-

Implementation Plan and Timeline (175 hours)

12-Week Implementation Timeline

Timeframe	Tasks and Deliverables
Week 1-2 (25 hours)	Environment Setup and Data Exploration • Set up development environment with necessary libraries • Explore sample EEG datasets to understand structure • Implement basic preprocessing pipeline for single-subject EEG • Deliverable: Working preprocessing script and data exploration report
Week 3-4 (35 hours)	EEG Preprocessing Pipeline • Develop comprehensive preprocessing pipeline for hyperscan data • Implement time-locking procedures for paired recordings • Create data validation tools • Deliverable: Documented preprocessing pipeline with validation metrics
Week 5-6 (40 hours)	CEBRA Model Adaptation • Modify CEBRA architecture for paired EEG inputs • Implement custom loss functions for dyadic data • Create training loops with validation procedures • Deliverable: Working CEBRA implementation for paired EEG data
Week 7-8 (35 hours)	Model Training and Validation • Train models on neurotypical dyads • Validate embedding quality through visualization • Fine-tune hyperparameters for optimal performance • Deliverable: Trained models and validation report
Week 9-10 (30 hours)	Clinical vs. Neurotypical Analysis • Apply trained models to clinical population data • Implement methods for comparing latent spaces • Identify discriminative features between populations • Deliverable: Analysis report with identified parameters
Week 11-12 (10 hours)	Documentation and Final Integration • Create comprehensive documentation • Package code for easy deployment • Prepare final report and demonstration materials • Deliverable: Complete pipeline with documentation and usage examples

Expected Outcomes and Societal Impact

This project has the potential to deliver significant technical tools and societal benefits:

1. **Software Deliverables:**

- A complete, documented Python package for preprocessing paired EEG data
- An adapted CEBRA implementation for analyzing dyadic neural recordings
- Analysis and visualization tools for latent space exploration

2. **Research and Clinical Applications:**

- Potential development of objective biomarkers for social communication disorders
- Tools for quantifying treatment efficacy in neurodevelopmental interventions
- Frameworks for understanding neural basis of social interaction across populations
- Possible applications in educational settings to measure student engagement

3. **Documentation and Knowledge Sharing:**

- Comprehensive user guide with code examples for the scientific community
- Technical documentation to support future extensions
- Final report connecting findings to the broader literature on social neuroscience

As noted by Hasson et al. (2022), developing tools that can objectively measure neural synchrony has profound implications for understanding and addressing social communication deficits. This project aims to contribute to this important goal by bridging advanced machine learning with clinical neuroscience.

Technical Skills and Qualifications

My background in ML and interest in BCI provide a foundation for this project:

- **Programming Skills:** Proficient in Python, with experience using ML libraries including TensorFlow, Keras, scikit-learn, and PyTorch
- **ML Experience:** Hands-on experience with various neural network architectures (CNN, LSTM) and training methodologies, as shown in previous projects with classification accuracy up to 98%
- **Data Analysis:** Experience with data preprocessing, feature selection, and visualization using libraries such as NumPy, Pandas, Matplotlib, and Seaborn
- **Cloud Computing:** AWS Cloud Practitioner and GCP Digital Leader certifications for potential scaling of computational resources
- **Relevant Coursework:** Completed coursework in machine learning, neural networks, signal processing, and data analysis

While expanding knowledge in neural data analysis specifically, this foundation provides an excellent starting point for the proposed work.

Motivation and Broader Impact

This project represents an opportunity to advance the scientific understanding of social communication through computational methods. The groundbreaking work by Dikker et al. (2021) demonstrating how neural synchrony can be measured and linked to social engagement has been particularly influential in shaping this proposal.

Potential Societal Impact

- **Clinical Applications:** Development of objective biomarkers could transform diagnosis and treatment monitoring for conditions like ASD, potentially enabling earlier interventions
- **Educational Implications:** Better understanding of neural synchrony has applications for measuring classroom engagement and developing inclusive educational strategies
- **Accessibility:** Open-source tools make advanced neural analysis accessible to broader research communities
- **Theoretical Advancement:** Contributes to fundamental understanding of social communication neuroscience

The techniques developed could eventually be extended to other BCI applications with clinical relevance, such as communication aids for non-verbal individuals. My interest in BCI aligns perfectly with this project's goals, which bridge technical innovation with meaningful societal impact.

References

1. Schneider, S., Lee, J.H., Mathis, M.W. (2023). *Learnable latent embeddings for joint behavioral and neural analysis*. Nature, 614, 487-493.
2. Sareen, E., Zahar, O., Ville, D. V. D., Gupta, A., Griffa, A., & Lemay, M. (2023). *Time-resolved modulation of directed coupling between brains and machines using EEG and a brain-computer interface*. Scientific Reports, 13(1), 1-13.
3. Papadopoulou, M., Friston, K., & Marinazzo, D. (2019). *Estimating directed connectivity from cortical recordings and reconstructed sources*. Brain Topography, 32(4), 741-752.
4. Hölle, D., Meekes, J., & Bleichner, M. G. (2021). *Mobile EEG and its potential to promote the theory and application of imagery-based motor rehabilitation*. International Journal of Psychophysiology, 160, 78-88.
5. Dikker, S., Michalareas, G., Oostrik, M., Serafimaki, A., Kahraman, H. M., Struiksma, M. E., & Poeppel, D. (2021). *Crowdsourcing neuroscience: Inter-brain coupling during face-to-face interactions outside the laboratory*. NeuroImage, 227, 117436.
6. Hasson, U., Jiang, J., Dumas, G. (2022). *Neural modeling of social communication: from brain regions to algorithmic implementations*. Trends in Cognitive Sciences, 26(3), 245-268.
7. Hasson, U., & Frith, C. D. (2016). *Mirroring and beyond: coupled dynamics as a generalized framework for modelling social interactions*. Philosophical Transactions of the Royal Society B: Biological Sciences, 371(1693), 20150366.

Communication and Collaboration Plan

I understand the importance of regular communication for project success. My plan includes:

- ✉ Weekly progress updates via email with summaries of completed work, challenges faced, and goals for the coming week
- 📺 Bi-weekly video meetings to discuss the project in depth and get feedback on my approach
- </> Regular code commits with detailed documentation
- 💬 Active participation in community discussions and code reviews
- 📅 Flexibility to adjust communication frequency based on mentor preferences

I value constructive feedback and am committed to maintaining a productive and collaborative relationship with my mentor and the broader ML4Sci community.

This project represents an opportunity to contribute to both the ML4SCI community and the field of neural data analysis, with potential far-reaching implications for understanding and addressing social communication disorders.
Thank you for considering this proposal.